The Seoul Report on Systems Research

LORENZO ALVISI*, BYUNG-GON CHUN, HAIBO CHEN, PETER CHEN, DONG DU, GERNOT HEISER, FRANS KAASHOEK, ANA KLIMOVIC, CHRISTOS KOZYRAKIS, BUTLER LAMPSON, JEANNA MATTHEWS, JEFF MOGUL, KYOUNGSOO PARK, DAVID PATTERSON, ROBBERT VAN RENESSE, TIMOTHY ROSCOE, DIMITRIS SKARLATOS, ION STOICA, YUBIN XIA, HARRY XU, DING YUAN, JUNFENG YANG, and MINGXING ZHANG

Systems researchers met at SOSP 2025 to reflect on our community's impact and to examine the research challenges created by AI, specialized hardware, large-scale infrastructure, and the growing difficulty of building reliable systems.

Additional Key Words and Phrases: systems research, operating systems, artificial intelligence, reliability, hardware–software co-design

ACM Reference Format:
Lorenzo Alvisi, Byung-Gon Chun, Haibo Chen, Peter Chen, Dong Du, Gernot Heiser, Frans Kaashoek, Ana Klimovic, Christos Kozyrakis, Butler Lampson, Jeanna Matthews, Jeff Mogul, KyoungSoo Park, David Patterson, Robbert van Renesse, Timothy Roscoe, Dimitris Skarlatos, Ion Stoica, Yubin Xia, Harry Xu, Ding Yuan, Junfeng Yang, and Mingxing Zhang. 2026. The Seoul Report on Systems Research. 1, 1 (August 2026), 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Systems research has repeatedly turned difficult technologies into dependable foundations. Operating systems made hardware usable across applications. Distributed systems made many machines serve as one. Virtualization, storage, networking, and cloud platforms carried these ideas into infrastructure used at global scale. The influence also runs in the other direction: deployment experience has exposed new failure modes, forced abstractions to evolve, and supplied the problems that keep our field relevant.

These achievements demonstrate that our community is strong. Yet the assumptions beneath many of them are shifting together. General-purpose performance scaling has slowed. Domain-specific accelerators and new memory technologies are fragmenting the hardware substrate. AI is becoming both a major systems workload and a tool for producing systems. Meanwhile, long software supply chains, wide cloud interfaces, and tightly coupled services make it harder to know what a system will do, why it failed, or whom to trust.

In October 2025, the authors of this report met in Seoul, South Korea, for the first ACM SIGOPS Strategic Workshop, held with SOSP 2025. The meeting brought together researchers and practitioners from operating systems, distributed systems, cloud infrastructure, architecture, reliability, and AI. Following the spirit of the 2015 SOSP History Day and the database community's periodic research reports, we looked backward for durable lessons and forward toward problems that may define the next five to ten years.

This report summarizes the themes that recurred across the workshop and the agenda that emerged from subsequent synthesis. It is intentionally selective. It does not survey the full breadth of systems research, nor does it claim that every participant preferred the same solution. Where the discussion revealed different instincts, we state the underlying tension as a research question for our community rather than resolve it by assertion.

*Organizers and participants, listed alphabetically by last name.

1 KEY INSIGHTS

- **AI makes systems reasoning more important, not less.** Models and agents can generate code, tune policies, and explore designs, but the value of their output depends on specifications, evaluators, and verification that capture the real objective.
- **Reliability is a property of the whole system.** Trustworthy components and reliable protocols do not automatically compose into an available service; deployment, performance, feedback loops, diagnosis, and persistent hardware faults belong in the reliability model.
- **The next stack needs both co-design and stable interfaces.** Specialization requires coordination across models, software, hardware, memory, and operations, while narrow and declarative interfaces remain essential for portability, evolution, and verification.
- **Our community's knowledge and practices are research infrastructure.** We need to preserve engineering judgment in an AI-assisted world, turn tacit craft into transferable principles, and make participation, responsibility, and face-to-face exchange part of how the field measures progress.

2 WHAT HAS CHANGED FOR SYSTEMS RESEARCH?

Three changes cut across nearly every workshop session. The first is a new hardware balance. The end of Dennard scaling moved performance growth away from faster general-purpose cores and toward parallelism and domain-specific architectures. AI has accelerated that movement, but arithmetic throughput alone is no longer a useful account of the machine. Memory bandwidth and capacity, data movement, energy, packaging, and the placement of data across heterogeneous memories increasingly determine performance and cost. The operating system and runtime must therefore decide not only when work runs, but where its computation and state belong.

The second change is the dual role of AI. AI workloads bring probabilistic quality, enormous synchronous jobs, rapidly changing models, and a growing collection of serving and agent components. At the same time, AI tools can now inspect and rewrite system implementations rather than merely tune a black box. This creates an appealing loop in which an agent proposes a design, evaluates it, and iterates. It also moves the hard part: if the task is underspecified or the evaluator rewards the wrong behavior, the loop can efficiently optimize the wrong system. Problem formulation, measurement, and verification become central systems contributions.

The third change is the scale of composition. Modern services combine hardware, firmware, compilers, kernels, libraries, distributed protocols, deployment systems, provider APIs, and learned components. Each layer can be reasonable in isolation while the whole remains fragile. A local loss of capacity can become a performance outage; a temporary overload can trigger a degraded state that persists; a rollout or mistaken diagnosis can do more damage than the initiating fault. Operations and maintenance can no longer be treated as the phase after design. They are where the behavior of the design is finally revealed.

These changes pull systems research in two directions. Efficiency pushes us down the stack, toward model internals, memory hierarchies, specialized devices, and deployment details. Longevity pushes us toward simple mechanisms, declarative descriptions, and stable interfaces that allow pieces to evolve independently. The central question is not which direction to choose, but which distinctions must cross an interface and which can safely remain hidden.

3 RESEARCH CHALLENGES

The changed landscape presents many opportunities. We organize the workshop's discussion into four areas in which systems expertise is especially relevant: co-designing systems and AI, achieving reliability across the full system, rebuilding stable interfaces and operating-system foundations, and making systems knowledge transferable.

3.1 Systems and AI Must Be Co-Designed

Our community should treat AI in three roles: as a workload whose infrastructure we must build, as a component inside systems, and as an instrument for systems research. The roles share mechanisms but not success criteria. A serving system must account for output quality as well as latency and throughput. A learned component inside an operating system still needs testing, monitoring, and repair even though its logic is encoded in model state rather than source statements. An agent that rewrites code must be judged against the intended system behavior, not merely against a convenient benchmark. Conflating the roles risks building a fast tool that solves the wrong problem.

Make evaluators first-class artifacts. As AI tools move from parameter tuning to white-box construction, evaluation becomes part of the design. Useful evaluators must represent correctness, performance, resource cost, robustness, and operational constraints without collapsing them into a misleading score. They must detect solutions that improve the measured result by changing semantics, such as dropping work. This calls for diverse and held-out workloads, faithful simulators, and specifications that expose why a proposed change works and where it fails. Traditional testing, monitoring, repair, and formal reasoning remain valuable, but they must be adapted to learned behavior and opaque model state.

Build accuracy-aware infrastructure. AI services replace a binary notion of correctness with choices that couple system performance and output quality. Quantization, caching, speculative execution, and continuous batching can improve efficiency, but numerical choices and even batch composition can change results. A production system therefore needs explicit quality objectives and mechanisms that navigate them together with latency, throughput, availability, and cost. The same stack must tolerate slow or failed accelerators and loss of a cloud provider. Agentic workloads add persistent state, tools, orchestration, and computer use before the field has agreed on the basic abstractions that tie them together.

Develop model-facing OS abstractions. Treating a model as an ordinary process preserves compatibility but hides opportunities for resource management and protection. Replacing the operating system with a model discards mechanisms that remain necessary. A more promising path is evolutionary: expose declarative interfaces suited to an agent's goals, manage KV-cache and other model state explicitly, and schedule across CPUs, GPUs, NPUs, memories, and remote services. Such integration widens the security boundary. An agent acting with a user's authority needs constrained delegation, privacy protection, and auditable control over both machine actions and access to models and data.

Optimize the workflow, not one device. The gap between AI demand and conventional hardware improvement makes co-design unavoidable. Model and algorithm designers need efficiency constraints early enough to shape their choices; system software must make new accelerators usable; and automated exploration should compare designs across the stack. The relevant cost is often outside the GPU: CPU work, DRAM capacity, storage, data movement, and tools invoked by agents can dominate parts of a workflow. Research that reaches production quickly demonstrates the opportunity, but it also warns against optimizing a single kernel while moving cost elsewhere.

This does not require every academic group to own a giant cluster. Simulation, emulation, focused testbeds, and careful study of inference or fine-tuning can expose architectural and systems problems at useful scale. Indeed, academia has a special role in escaping the hardware lottery: today's stack is deeply optimized around dominant models and operations, which makes alternatives appear impractical before their software exists. Intermediate representations, compilers, runtimes, and evaluation tools can let a new hardware idea run meaningful workloads without first reproducing an entire ecosystem.

3.2 Reliability Must Span the Full System

Abstraction allows us to use a component without understanding its implementation, but not without depending on it. Trust passes transitively through libraries, compilers, operating systems, hardware, manufacturing, models, training code, and the tools that produced them. The right question is therefore not whether a component is trusted in the abstract, but what risk it creates in a particular system: how likely it is to fail or be compromised, and how much control that failure grants. Least privilege, isolation, focused verification, diversity, and redundancy reduce risk by limiting control or avoiding a single point of trust.

Treat availability as behavior, not a protocol property. Our field has reliable protocols, yet services built from them still fail. Availability includes completing useful work at acceptable performance, not merely returning a correct answer eventually. A fault-tolerant component that loses capacity can trigger queues and timeouts in its callers. A single slow participant can stall a synchronous AI job. Configuration changes and staged rollouts alter the system while it is running, and a plausible but wrong diagnosis can prolong the outage. We need models, experiments, and tools for deployment, observability, diagnosis, and repair as seriously as we need another protocol refinement.

Understand failures created by composition. Metastable failures show why component-wise reasoning is insufficient. A temporary trigger can push a service into a degraded state that persists after the trigger disappears. Retry amplification is one example, but a broader mechanism arises when individually stable components change one another's operating environments and become mutually destabilizing. We need compositional models that say when local control loops preserve global stability, how to detect that the system has crossed into a metastable state, and how to recover without repeating the same feedback.

Create practical beachheads of correctness. Whole-system verification remains difficult, but waiting for an all-or-nothing proof leaves production code untouched. Gradual verification starts with the state, invariants, functions, and code paths that matter most, then finds a focused bug or proves a focused property. The verified region can grow as the system changes. This approach is especially valuable when combined with risk analysis: proof effort can be directed toward components whose failure grants the greatest control or produces the least recoverable harm.

Design above unreliable hardware. Persistent and silent hardware faults weaken the assumption that restart returns execution to a sound base. A mercurial core may reproduce an error after checkpoint recovery, while silent corruption can escape conventional failure detectors. Software-defined checking and redundancy can use otherwise idle resources to detect or mask these faults. The research challenge is to obtain meaningful coverage at controlled overhead and to connect hardware evidence to the end-to-end correctness claims made by the service.

3.3 The Stack Needs Stable Interfaces and Better Foundations

Specialization often rewards vertical integration: an application, runtime, cloud service, and accelerator can be tuned together for immediate gains. Over time, however, a tightly coupled stack can obstruct portability, independent innovation, and verification. Modern cloud platforms illustrate the danger. Instead of a narrow waist with many

implementations below and applications above, they expose a wide and changing collection of provider-specific APIs. AI-generated software magnifies the problem because every additional interface expands what the generator must understand and what the evaluator must test.

Find narrow waists through experience. Stable interfaces such as those in earlier operating-system, network, and data-management stacks supported independent evolution because they exposed a small common mechanism. The cloud needs similar waists, but we should not assume that a committee can design the right one before enough systems exist. Narrow waists often emerge after experimentation reveals what is truly common. Research can accelerate that process by comparing real services, identifying repeated mechanisms, and testing whether a candidate interface provides portability without suppressing important optimizations.

Prefer declarative interaction where possible. An imperative sequence forces the caller to coordinate intermediate state and often encodes accidental details of one implementation. A declarative request states the desired outcome and leaves room for the system to optimize, recover, and choose among heterogeneous resources. This idea connects model-facing interfaces, cloud portability, configuration, and operations. It also creates an obligation: the declaration must be precise enough to verify, and the system must expose enough explanation to show how the outcome was achieved.

Revisit principled OS design. Operating systems are not a solved foundation. A small verified kernel can provide strong low-level guarantees and still be too close to an assembly language for most system builders. We need usable structure above that base without recreating an unverifiable monolith. Radical simplicity, fine-grained modules, separation of concerns, sequential event-driven logic within modules, and replaceable use-case-specific policies offer one path. The goal is not simplicity as an aesthetic; it is a system whose mechanisms can be understood, verified, maintained, and adapted without giving up performance.

The tension between co-design and stable interfaces should remain visible. A waist that hides every distinction prevents efficient placement across accelerators and memories. An interface that exposes every device detail simply moves the wide waist upward. Our task is to identify the smallest information that permits good placement, protection, and diagnosis. Classic systems design advice still applies: state goals clearly, divide a hard problem into smaller ones, evolve incrementally, and settle for a good-enough mechanism when greater generality would make the system impossible to understand or operate.

3.4 Systems Craft Must Become Transferable Knowledge

Much of what makes a strong systems researcher effective is tacit: how to choose a boundary, interpret an anomalous measurement, simplify a design, debug across layers, or recognize that an optimization will fail in operation. This knowledge moves through apprenticeship and is repeatedly rediscovered in individual projects. Our field produces many systems, but it is less systematic about extracting the principles that survive them. The result is a growing related-work burden, fragile teaching material, and insights tied to products or hardware that may disappear before the next generation can use them.

We should make synthesis a first-class research contribution. Surveys, reference models, shared artifacts, negative results, and abstractions validated across multiple systems can reveal where results agree and where design spaces remain unexplored. This is not an argument to stop building. Concrete systems are how we discover behavior and test ideas. The change is to treat the reusable knowledge extracted from them as an intended output rather than a by-product.

AI makes this need more urgent. Code generation can increase productivity, but generated code does not teach a student why the boundary is correct or how to recognize a plausible failure. Education should use the tools while strengthening problem formulation, evaluation, and explanation. Until verification itself is dependable, the ability to inspect and challenge an automated result remains a core systems skill. A codified body of systems knowledge can also improve the tools: it supplies specifications, design alternatives, and failure patterns that are more durable than examples scraped from yesterday's code.

4 COMMUNITY

Technical priorities alone do not determine the community's future. Who participates, what our venues reward, how we educate new researchers, and whether builders account for the consequences of their systems all shape which problems receive attention and which lessons survive.

Use conferences for interaction. Publication and community building are related but different functions. Review can evaluate a paper without reproducing the value of being together. Conferences should preserve time for practitioners, tutorials, focused discussions, early ideas, and exchange across seniority and research areas. Workshops and experimental tracks can test new formats, but their outcomes should be observed rather than defended as tradition. A larger publication pipeline is not, by itself, a stronger community.

Match participation with representation. The geographic center of systems research is broadening. Workshop data comparing 2016–2020 with 2021–2025 found that the average number of papers per conference from Asian institutions rose from about 6 to 15.2, while active faculty and contributing institutions also grew sharply. Yet program committee representation remained about 11.3%, below a publication share approaching 30%. The precise measures deserve continued scrutiny, but the direction is clear: governance and service should broaden with research output. We should define the community around people who build and study computing infrastructure, not around loyalty to a particular venue or region.

Keep judgment and sustainability in the curriculum. Students should learn to use AI tools without surrendering the responsibility to explain and verify their output. Education can emphasize problem statements, invariants, evaluation, and operational reasoning alongside implementation. The fast replacement cycle of AI hardware creates another systems opportunity: aging accelerators may still serve other workloads, extending their useful life and reducing embodied-carbon cost. Resource efficiency should include the lifetime of the machine, not only the utilization of its newest device.

Accept responsibility for system effects. Systems are not neutral plumbing. Their interfaces and defaults distribute power, determine what can be observed, and influence whether a harmful decision can be challenged. System builders should consider observability, provenance, explanation, access, redress, redundancy, and independent sources of truth as technical mechanisms. Safeguards should reflect both the likelihood and severity of harm, especially when recovery is impossible. Policy is not solely our task, but our knowledge of the system gives us a responsibility to make its risks and control points visible.

5 CONCLUSION

Systems research is entering a period in which its workload, substrate, and design process are changing together. AI creates new infrastructure demands and new ways to explore designs. Specialization makes hardware software co-design necessary. Scale exposes failures that no component model captures. These forces reward work close to

implementation, but they also increase the value of simple mechanisms, stable interfaces, and knowledge that survives a particular artifact.

Our community already has the habits needed for this transition: expose a useful abstraction, characterize a tradeoff, build the mechanism, measure the whole system, and revise the design when reality disagrees. The agenda for the next five to ten years is to apply those habits across models, hardware, software, operations, and human institutions. If we do so, systems research can make rapid technological change not only faster, but more understandable, dependable, and open to improvement. We thank the speakers, panelists, session chairs, organizers, and attendees of SSW '25 for the talks and discussions on which this report is based. The workshop program and recordings are available at https://ipads.se.sjtu.edu.cn/sigops-strategic/.

REFERENCES

[1] David H. Abernathy, John S. Mancino, Charls R. Pearson, and Dona C. Swiger. 1973. Survey of design goals for operating systems. *SIGOPS Oper. Syst. Rev.* 7, 3 (July 1973), 19–34. https://doi.org/10.1145/775260.775263

[2] Jeffrey Dean and Luiz André Barroso. 2013. The tail at scale. *Commun. ACM* 56, 2 (Feb. 2013), 74–80. https://doi.org/10.1145/2408776.2408794

[3] Hadi Esmaeilzadeh, Ting Cao, Xi Yang, Stephen M. Blackburn, and Kathryn S. McKinley. 2012. Looking back and looking forward: power, performance, and upheaval. *Commun. ACM* 55, 7 (July 2012), 105–114. https://doi.org/10.1145/2209249.2209272

[4] Butler Lampson. 2000. Computer systems research (invited talk) (summary only): past and future. *SIGOPS Oper. Syst. Rev.* 34, 2 (April 2000), 8–9. https://doi.org/10.1145/346152.346162

[5] Butler W. Lampson. 1983. Hints for computer system design. *SIGOPS Oper. Syst. Rev.* 17, 5 (Oct. 1983), 33–48. https://doi.org/10.1145/773379.806614

[6] C. Mohan. 1978. Survey of recent operating systems research, designs and implementations. *SIGOPS Oper. Syst. Rev.* 12, 1 (Jan. 1978), 53–89. https://doi.org/10.1145/775323.775329

The following authors served as editors of this article (Haibo Chen also served as corresponding author):

Haibo Chen (haibochen@sjtu.edu.cn) is a Distinguished Professor in the School of Computer Science at Shanghai Jiao Tong University and Director of the Institute of Parallel and Distributed Systems.

Ding Yuan (yuan@eecg.toronto.edu) is a Professor in the Department of Electrical and Computer Engineering at the University of Toronto.

KyoungSoo Park (kyoungsoo@gmail.com) is a Professor in the Department of Computer Science and Engineering at Seoul National University.

Dong Du (Dd_nirvana@sjtu.edu.cn) is an Associate Professor in the Institute of Parallel and Distributed Systems at Shanghai Jiao Tong University.